

ROAR Current-State Scenario Map

Date: 2026-03-31

Repo snapshot basis: current local code plus workflow JSON in `n8n/workflows/`

1. Executive Summary

This document maps what the app currently handles in code, how each case is verified and routed, where the system already eliminates unnecessary manual work, and where a future image-proof feature should fit.

The current ROAR system already reduces unnecessary work in four main ways:

- It forces structured intake instead of fully freeform customer reporting.
- It extracts live data from OMS, payments, refunds, shipments, inventory, and duplicate-check sources before triage.
- It uses deterministic and/or workflow-driven criteria to decide autonomous handling, approval-required handling, or escalation.
- It keeps human work mostly for edge cases: high-value refunds, failed rule checks, rejected approvals, and escalations.

The current system is functional, but there are also important mismatches between docs, workflow JSON, and some live workflow configuration. Those mismatches are listed explicitly in Section 10.

2. End-to-End Architecture

Current architecture in code:

- Frontend (Next.js)
- FastAPI backend
- n8n workflows
- FastAPI internal/public endpoints
- Database

Current high-level runtime flow:

1. Customer logs in and opens `/chat`.
2. Customer selects an order, issue subtype, and optionally resolution preference.
3. Frontend sends `POST /cases`.
4. Backend creates the case and triggers WF1.
5. WF1 asks intake questions or classifies the dispute as ready for WF2 / insufficient context / escalated.
6. WF2 extracts data from internal data-source endpoints and stores an `information\_bundle` on the case.
7. WF3 triages the case into one of the following outcomes:
 - autonomous or approved-executing path
 - awaiting approval
 - escalated human required
8. WF4 creates an escalation summary for human handling.
9. WF5 executes approved resolution plans.
10. WF6 creates a final case report on close.

3. Customer Intake Scenarios Currently Supported

Top-level dispute types

The system currently only persists these top-level dispute types:

- `refund`
- `delivery`

This is intentional for compatibility with the current backend and workflows.

Intake subtypes currently supported in backend validation

The backend currently accepts these `dispute\_subtype` values:

- `package\_never\_arrived`
- `delivery\_late`
- `wrong\_delivery\_address`
- `damaged\_goods`
- `wrong\_item`
- `quality\_issue`
- `partial\_fulfillment`
- `duplicate\_charge`
- `return\_for\_refund`
- `other`

Resolution preference currently supported

The backend currently accepts these `resolution\_preference` values:

- `refund`
- `replacement`

This field is currently required only for these subtypes:

- `damaged\_goods`
- `wrong\_item`
- `quality\_issue`

4. Current Intake UX and Structured Follow-Up

Initial intake screen

Current intake collects:

- Order
- Issue subtype
- Conditional resolution preference

This means the system already captures more precise intent than just `refund` or `delivery`.

Structured follow-up questions currently supported on the frontend

The current frontend can render structured chip responses for:

- issue-detail clarification
- refund receipt question
- refund payment method question

- refund timing question
- delivery tracking question
- delivery SLA question
- delivery carrier-contact question
- item selection question
- amount confirmation question
- order confirmation question
- tracking confirmation question
- estimated-delivery confirmation question

Item-level follow-up

The frontend now supports item-level multi-select chips from live order data when RAI asks questions like:

- which items were missing
- which items were damaged
- which items were wrong
- what items were affected

This is important because it lets the system reason at item level for partial receipt, damaged-item, and wrong-item scenarios.

5. Current Data Sources Extracted by the System

The app already performs systematic extraction from backend data sources rather than relying only on customer text.

Sources available through internal endpoints

Current internal source endpoints include:

- Orders
- Transactions / payments
- Prior refunds
- Shipments
- Inventory
- Duplicate-check

Information bundle fields currently assembled

The current workflow/data retrieval layer can assemble a case `information\_bundle` that includes:

- order data
- order items
- payment / transaction data
- refund history
- shipment data
- tracking events
- stock records
- dispute type
- dispute subtype

- resolution preference

Why this reduces unnecessary work

This already removes a large amount of repetitive human investigation because the system does not need a human to manually check:

- whether the order exists
- whether payment was confirmed
- whether the order was fulfilled/returned
- whether shipment exists and what its status is
- whether duplicate refund activity exists
- whether inventory exists for replacement-related logic

6. Current Case Statuses and Transition Surface

Current statuses in backend code:

- ``pending_triage``
- ``awaiting_approval``
- ``approved_executing``
- ``rejected_human_required``
- ``escalated_human_required``
- ``resolved``
- ``closed``

Current valid transitions in backend code

- ``pending_triage`` -> ``pending_triage``, ``awaiting_approval``, ``escalated_human_required``, ``closed``
- ``awaiting_approval`` -> ``approved_executing``, ``rejected_human_required``, ``closed``
- ``approved_executing`` -> ``resolved``, ``closed``
- ``rejected_human_required`` -> ``resolved``, ``closed``
- ``escalated_human_required`` -> ``resolved``, ``closed``
- ``resolved`` -> ``closed``
- ``closed`` -> none

Current timeout behavior in backend code

Current timeout service applies only to:

- ``pending_triage``

This means:

- AI intake can auto-timeout on customer inactivity.
- Escalated, approval, and post-resolution cases should not auto-close from the current timeout service.
- ``resolved`` should still be explicitly closed once the conversation is done.

7. Resolution and Escalation Criteria Currently in Use

This section describes the criteria the app currently uses or attempts to use based on backend code and workflow JSON.

7.1 Refund-type verification logic

Current refund verification data points used by the system:

- payment confirmation status
- order fulfillment / returned status
- transaction amount
- prior refund activity / duplicate disputes
- transaction / order age window
- customer-provided context from intake and follow-up questions

Current refund-oriented deterministic checks observed in code/workflow JSON

The current system uses some variation of these checks:

- payment must be confirmed
- order should be fulfilled or returned for normal refund processing
- amount threshold determines whether approval or escalation is needed
- duplicate history can block or change the path
- time window from order date matters

Important current mismatch

There is a current mismatch between materials:

- business-rule docs describe a 7-day refund window in some places
- repo workflow JSON for WF3 contains a 30-day return-window style rule
- some live workflow screenshots previously showed 7-day logic in deterministic triage

This needs normalization if you want one defensible rule source.

7.2 Delivery-type verification logic

Current delivery verification data points used by the system:

- shipment existence
- shipment status
- tracking events
- SLA/EDD timing
- duplicate complaint history
- customer-provided tracking and receipt context

Current delivery-oriented deterministic checks observed in workflow JSON

The current system uses some variation of these checks:

- shipment delayed and not terminally lost/failed
- item shipped
- no duplicate complaint
- within SLA-related rule window

7.3 Approval-required path

Current approval-required path is used when refund logic passes into a human-approval threshold rather than immediate autonomous execution.

Observed current examples:

- refund cases set to `awaiting\_approval`
- approver can approve or reject
- approval can trigger WF5 execution or create/refine refund handling depending on the branch

7.4 Escalation path

Escalation is currently used when automated verification or deterministic rules fail, or when the system determines human intervention is required.

Observed current escalation-style triggers include:

- payment not confirmed
- order not fulfilled
- amount/rule threshold exceeded
- age window exceeded
- insufficient context branch from intake
- post-approval rejection / human-required branches

8. Scenario Map: What the App Currently Can Handle

A. Package never arrived

Likely top-level type:

- `delivery`

Current system can do:

- ask tracking-status follow-up
- ask estimated-delivery-passed follow-up
- ask carrier-contact follow-up
- compare against shipment/tracking data
- route to triage

Human work eliminated by system:

- shipment lookup
- tracking lookup
- duplicate-check lookup
- SLA-related verification

Escalate when:

- shipment evidence conflicts
- rule checks fail
- workflow routes to human-required branch

B. Delivery is late

Likely top-level type:

- `delivery`

Current system can do:

- ask tracking-status follow-up

- ask whether estimated delivery passed
- ask whether carrier was contacted
- compare against shipment and tracking data

Human work eliminated by system:

- manual shipment inspection
- manual ETA lookup
- manual duplicate complaint check

C. Wrong delivery address

Likely top-level type:

- `delivery`

Current system can intake this subtype.

Current uncertainty:

- this subtype is accepted by backend intake validation, but there is no strong evidence in the current frontend structured-response set or deterministic triage rules that it has a dedicated special-case logic branch.

Current likely behavior:

- treated as a delivery case with general follow-up and standard triage data extraction
- may escalate if automation cannot determine a safe resolution path

D. Damaged goods

Likely top-level type:

- usually `refund`, possibly replacement-preference driven

Current system can do:

- capture subtype `damaged\_goods`
- capture `resolution\_preference`
- ask for more issue detail
- ask which items were damaged when only some items are affected
- ask payment method when still relevant
- use order item data for structured item selection

Human work eliminated by system:

- order-item lookup
- payment lookup
- duplicate lookup
- structured item identification instead of freeform agent clarification

Current escalation / resolution behavior:

- may go to approval or escalation depending on deterministic checks and thresholds
- replacement preference is now captured, but actual replacement execution logic is not yet proven as fully operational across the whole workflow chain

E. Wrong item / wrong variant

Likely top-level type:

- currently often normalized into `refund` plus subtype `wrong\_item`

Current system can do:

- capture subtype `wrong\_item`
- capture `resolution\_preference`
- ask for more issue detail
- ask which items were wrong/affected when only part of the order is affected
- use item-level structured selection

Human work eliminated by system:

- manual item identification
- manual order lookup
- manual transaction lookup

F. Quality issue

Likely top-level type:

- currently normalized into `refund` plus subtype `quality\_issue`

Current system can do:

- capture subtype `quality\_issue`
- capture `resolution\_preference`
- ask issue-detail follow-up
- use refund/payment verification path

Current gap:

- there is less direct evidence that quality-specific deterministic triage rules are fully distinct from general refund logic. It appears mostly subtype-informed, not a fully separate policy engine path.

G. Partial fulfillment / missing items

Likely top-level type:

- often `refund`

Current system can do:

- capture subtype `partial\_fulfillment`
- ask whether any part of the order was received
- ask item-level question when only part of the order was received or affected
- show all order items as structured multi-select chips

This is one of the clearest examples of unnecessary-work elimination because the system can now:

- pull the full order item list
- ask the customer to identify affected items directly
- avoid forcing a human to inspect and reconcile vague descriptions manually

H. Duplicate charge

Likely top-level type:

- likely `refund`

Current system can do:

- capture subtype `duplicate\_charge`
- use duplicate/refund/payment-related data sources

Current gap:

- there is not yet strong evidence of a fully subtype-specific frontend follow-up flow for duplicate charge beyond the general refund path. It is structurally supported, but not deeply specialized.

I. Return for refund

Likely top-level type:

- `refund`

Current system can do:

- capture subtype `return\_for\_refund`
- may create return-request records during human handling
- supports return request creation and approve/reject endpoints on backend

Human work eliminated by system:

- return request persistence
- status transitions for return request approval/rejection
- customer-facing system messaging on return request actions

J. Other

Likely top-level type:

- `refund` or `delivery` depending on intake path

Current system can do:

- accept the subtype
- fall back to free text
- proceed through workflow intake and likely escalate if structured automation cannot safely route the case

9. How the App Currently Eliminates Unnecessary Work

The system already eliminates unnecessary work in these specific ways.

9.1 Intake standardization

Instead of allowing only open-ended customer complaints, the app forces:

- order selection
- issue classification
- optional resolution preference
- structured chip responses when RAI asks supported questions

This reduces agent time spent translating vague descriptions into policy-relevant categories.

9.2 Automatic data-source extraction

The app already extracts or can extract:

- order state
- payment confirmation

- shipment state
- tracking data
- inventory data
- refund history
- duplicate-risk information

This removes repeated manual back-office verification work.

9.3 Item-level disambiguation

For subset-of-order problems, the app can now ask which items were affected and present the real order items as chips.

This removes unnecessary human work in cases like:

- only one item was wrong
- only some items were damaged
- only some items arrived

9.4 Deterministic threshold handling

The triage layer is designed to move easy cases forward and isolate human work for exceptions.

Examples:

- threshold-based approval requirements
- escalation for failed rule checks
- duplicate/refund-history-informed handling

9.5 Human work is reserved for higher-friction cases

Current human-required cases include:

- escalations
- rejected approvals
- approval-gated refunds
- cases where deterministic checks fail
- cases where customer context remains insufficient or conflicting

10. Current Gaps, Drift, and Operational Risk Areas

This is the section you should use to confirm whether the app is doing what you think it is doing.

10.1 Docs vs current code drift

There are meaningful mismatches between some docs and current implementation.

Examples:

- inactivity timeout rules differ from what some docs describe
- refund-window logic differs between docs, repo workflow JSON, and some live workflow screenshots
- some subtypes are structurally supported but not fully specialized in workflow logic

10.2 Workflow configuration drift risk

Some routing behavior depends on live n8n nodes, not just repo JSON.

A previously observed example:

- `Route on Decision` in live WF3 referenced `routing\_decision` while the upstream node output was `triage\_decision`
- that kind of mismatch can make the workflow appear "stuck" even when the logic node succeeded

10.3 Replacement path maturity

The app now captures `resolution\_preference = replacement`, but the entire downstream replacement execution path is not yet as clearly proven end-to-end as refund handling.

That means:

- intake supports replacement intent
- some triage context supports replacement-aware reasoning
- but the replacement plan/execution path should still be treated as an area to verify carefully

10.4 Some subtype-specific policies are still shallow

Subtypes like:

- `wrong\_delivery\_address`
- `duplicate\_charge`
- `quality\_issue`
- `other`

are accepted by the backend and intake, but current evidence suggests they still lean on broader refund/delivery handling rather than highly specialized policy branches.

11. Where Image Proof Should Fit

A customer-uploaded image-proof feature is a good fit for this system, but only if it is inserted at the right layer.

Recommended placement in the pipeline

Image proof should fit after initial intake classification but before final triage decision, as part of the evidence-gathering layer.

Recommended sequence:

1. Customer selects order, issue subtype, and optional resolution preference.
2. RAI asks structured clarification questions.
3. If the subtype or follow-up indicates a visual-verification case, prompt for image upload.
4. Backend stores the image and attaches machine-readable evidence results into the case `information\_bundle`.
5. WF2/WF3 consume that evidence as another data source during verification and triage.

Cases where image proof would add the most value

Highest-value use cases:

- damaged item
- damaged package
- wrong variant / wrong color / wrong size
- item not matching listing
- partial fulfillment when the package contents can be shown

What image proof should produce

Do not treat image upload as just an attachment bucket. It should produce structured evidence such as:

- package damaged: yes/no/uncertain
- item damaged: yes/no/uncertain
- wrong variant likely: yes/no/uncertain
- visible item count mismatch: yes/no/uncertain
- confidence score
- extracted notes / rationale

Recommended storage shape

The cleanest place is inside the case evidence bundle, for example conceptually:

- ``information_bundle.evidence.images[]``
- ``information_bundle.evidence.visual_verification``

Example conceptual fields:

- uploaded image metadata
- verifier result
- verifier confidence
- verifier tags such as ``damaged_package``, ``damaged_item``, ``wrong_variant``, ``partial_contents``

Why this fits the current architecture

This preserves the current architecture correctly:

- frontend uploads evidence to FastAPI
- FastAPI stores and/or preprocesses the file
- workflow/backend verification consumes the result
- no direct frontend-to-workflow shortcut

Why this reduces unnecessary work further

If implemented properly, image proof can eliminate another chunk of unnecessary work by reducing human review for cases where the visual evidence is strong enough to support:

- valid damage confirmation
- likely wrong-item confirmation
- likely replacement eligibility
- likely escalation necessity when evidence conflicts with extracted backend data

12. Confirmation Checklist: What the App Currently Does Well vs What Still Needs Verification

Already clearly strong

- structured intake
- live order selection
- subtype capture
- conditional replacement vs refund preference capture
- backend validation of intake combinations
- data extraction from multiple backend sources

- item-level structured selection for affected-item cases
- approval and escalation role separation
- persisted refund and return records
- customer-visible system messaging on many backend actions

Still needs deliberate verification in your environment

- exact live WF1 prompt contents
- exact live WF3 routing-node field mapping
- final refund-window rule source of truth
- replacement end-to-end execution path
- subtype-specific handling depth for `wrong\_delivery\_address`, `duplicate\_charge`, `quality\_issue`, `other`
- whether all customer-facing messages fire exactly when expected in every branch

13. Bottom Line

Yes: the app already eliminates a meaningful amount of unnecessary work through structured intake, live data-source extraction, deterministic verification, and triage-driven routing.

It is not yet a perfectly unified policy engine because there is still some drift between docs, repo workflow JSON, and live workflow configuration.

The strongest current proof of systematic work elimination is this chain:

- structured issue capture
- order-level and item-level grounding
- live backend data extraction
- deterministic rule checks
- approval/escalation only for exceptions

The highest-value next step is image-proof verification inserted into the evidence layer before triage. That would extend the same design principle already used for orders, payments, shipments, refunds, and inventory: collect objective evidence first, then minimize manual work.